

Speaker Companion

Schema drift, caught — the DCM demo. One entry per slide. Read the **amber** aloud; the grey is your backup for questions.

How to use this. Glance down, read the amber **SAY** lines in your own voice — they're written to be spoken, not recited. The **(beat)** markers are where to pause or click. The grey **DETAIL** is what to reach for if someone asks — numbers, the “why”, the sharp follow-ups. You will not need most of it; it's there so you're never caught out.

☐ say out loud ☐ backup / if asked

01 The database that couldn't see itself

TITLE

SAY

This is about a gap in how we manage our Snowflake schema — and a native way to close it.

The one-line version: our repo can always **build** the database. Until now it could never prove the database still **matches** it. **(beat)** I'll show you what that gap costs, and then I'll close it live.

IF ASKED

- “DCM” = **Declarative Change Management** Projects, a Snowflake feature that went **GA on 7 Aug 2026**.
- This is a proof of concept in a personal account — 8 tables, 53 columns, no production data.

02 Our pipelines cannot see the database

THE PROBLEM

SAY

Our pipelines create tables with `CREATE TABLE IF NOT EXISTS` — 70 of these, across 8 files.

Here's the catch. If the table already exists, that statement does **nothing**. It doesn't look inside.

(beat) So someone adds a column by hand on a Tuesday, the pipeline re-runs, and Snowflake says “already exists, statement succeeded.” Green. Honest. And completely blind to the extra column.

No data is harmed, nothing breaks — which is exactly why it stays invisible for months.

IF ASKED

- “Does it recreate the table each run?” — **No**. First run creates; every run after is a no-op. It exists so a fresh environment builds.
- Snowflake isn't lying — it tells you it did nothing. The defect is that “**did nothing**” **counts as success**, and nothing compares the table to what we declared.

SAY

Two different promises here. “The repo can build the database” — always true. “The database still matches the repo” — that was never true, and nothing could tell us the difference.

(beat) For context: a dashboard in our estate sat frozen for **seven months** behind a check that looked like a check and never checked anything. Different bug — but the same shape.

IF ASKED

- By “repo” here I mean the **Matillion pipeline repo** that holds the 70 DDL statements — not the DCM repo.
- The dashboard freeze was a trailing-comma syntax error hidden by an alert-then-succeed pattern. Recorded in our findings.

04 Declare the state. Snowflake finds the difference.

SAY

DCM Projects is native to Snowflake — no external tool, no state file. It went GA a few weeks ago.

Instead of writing instructions, you **declare** what a table should look like — `DEFINE`, not `CREATE` — and Snowflake works out the difference. **PLAN** is a dry run that changes nothing. **DEPLOY** applies it.

(beat) And if you delete a table's definition, the next deploy drops the table. The file **is** the source of truth.

IF ASKED

- “vs Terraform?” — Terraform suits account-level objects and keeps an external state file; DCM is native and stateless. Common practice is both.
- Tables, views, tasks, roles, warehouses, schemas are all GA within DCM.

05 The DDL stops living inside the ETL

SAY

Today our DDL lives inside the ETL — it runs as a side effect of a data load. No review, no diff, and drift is invisible.

With DCM, the DDL lives in git, reviewed like any code, and the plan shows the impact before anything lands. **(beat)** The database stops being a place things quietly accumulate, and becomes a build output.

IF ASKED

- One command rebuilds an entire environment from the repo — useful for onboarding and for standing up a fresh dev.

SAY

The structure is fixed — definitions must sit under `sources/definitions`, or Snowflake simply won't read them.

The manifest says **where** to deploy and **what varies** between environments. That's the promotion story: the same files build dev and prod, and `env_suffix` — `_DEV` or `_PROD` — is the only difference. *(beat)* Promoting to prod is just pointing the same reviewed file at a different config. You never keep a second copy of the DDL.

IF ASKED

- “Promotion is pointing at a target” = same definition files, run with `USING CONFIGURATION PROD`, which appends `_PROD`. No copy of the scripts to drift apart.
- The alternative everyone lives with: a separate prod copy of the DDL that slowly diverges until nobody can reproduce prod.

07 How it fits together

SAY

Here's the whole thing. GitHub is the source of truth. Snowflake keeps its own clone. The DCM project deploys prod **from that clone** — from git, never from someone's laptop.

And every night at 5, a task runs **PLAN** against prod, logs the result, and emails us only if something's off. *(beat)* PLAN is automated because it only reads. DEPLOY is a human decision — because it drops columns to fix things.

IF ASKED

- The clone is a `GIT REPOSITORY` object; it updates only on `FETCH`, which the nightly job runs before it plans.
- The monitor watches **prod** — the thing that must match git.

08 Three copies. All can be stale.

SAY

One thing that trips everyone up: there are three copies of these files, and all three can be stale independently. GitHub, Snowflake's clone, and my personal workspace.

A clone is a snapshot — it knows nothing about a new commit until you `FETCH`. *(beat)* GitHub is authoritative. The other two are caches.

IF ASKED

- A workspace lives in a per-user database — `SHOW GIT REPOSITORIES` won't even list it, and a scheduled task can't read it. That's why the account-level clone exists.

SAY

Changes only travel one way — from git, down to the database. A hand-made `ALTER` never flows back into the repo. There's no path back.

(beat) That asymmetry is the entire reason drift detection has to exist. If everything went through git, you wouldn't need it.

IF ASKED

- “Why can't DCM read prod and update the repo?” — because there's no safe way to tell an intended hand-change from a mistake. Reverse-engineering the database into declarations is guessing; detecting the difference and letting a human decide is not.

10 Dev fast. Prod from git.

SAY

So how does a developer actually work? Dev is deployed straight from the workspace — fast, no gate. Prod is deployed **from git**, after review, with `USING CONFIGURATION PROD`.

Same files build both. And because prod comes from git, git and prod agree by construction — **(beat)** which is what keeps the nightly check honest. If I deployed prod from my laptop and pushed afterwards, the check would flag my own deliberate change as drift.

IF ASKED

- Dev and prod are two databases in the **same account**, separated by `env_suffix`. No second account, no paid upgrade — just compute credits.
- Real enterprises often split dev/prod across separate accounts — a target each. Same idea.

11 Three questions. Git answers two.

SAY

This is the spine of the whole idea. Three questions.

What changed in the **definition**? — `git diff` answers that. What will change in the **database**? — the plan answers that. What changed **without going through either**? **(beat)** That third one — that's drift. And it's the one we have never been able to see.

IF ASKED

- “Isn't CI enough?” — CI only sees changes that go *through* CI. A hand-made `ALTER` on prod never touches a pull request, so no amount of CI catches it. That's the third diff.

SAY

The drift check has three verdicts, not two. **Clean** — matches the repo. **Drift** — someone changed something and it's revertible; the check names the exact column. And **Error** — the change can't be auto-reverted, and it may be hiding other drift.

Why three? *(beat)* Because 45 of our 53 columns aren't the last column in their table, and Snowflake can't reorder columns — so a dropped column usually produces Error, not Drift. A two-state monitor would read the worst case as a broken job.

IF ASKED

- Un-revertible cases: dropping a non-last column, narrowing a VARCHAR, renaming. Recovery is unload → drop → redeploy → reload — planned maintenance, not a 3am fix.

13 It names the column · what Snowflake forgets

SAY

When drift is found, the plan names the column, its type, and which way the fix goes — not just “this table differs.” That's the difference between an alert worth waking for and one people mute.

And here's a surprise: Snowflake records every **deploy**, but **no plans at all**. The drift check is the one thing it doesn't remember. *(beat)* So we keep our own log — which answers “when did this drift start,” the question you actually ask in an incident.

IF ASKED

- Measured on the account: 7+ plans, 4 deploys → history shows 4 deploys, zero plans. No `ACCOUNT_USAGE` view either. Re-checked after GA — still true.

14 Thirteen findings. Four from deliberate sabotage.

SAY

We found thirteen things, and four of them came from deliberately breaking it.

The verdict — yes, it detects hand-made drift, at column level. It records deploys but never plans, so an audit table is mandatory. Dropping a non-last column is un-revertible. And my favourite: *(beat)* our own monitor sat suspended for a day while its health view cheerfully reported “OK.”

Every failure we found was the same shape — something reporting success without checking what it claimed to check.

IF ASKED

- “Doesn't finding bugs in your own monitor undermine it?” — the opposite. We found them *because* we deliberately broke it and re-ran end to end. A monitor nobody has made fail is one nobody should trust.

SAY

Let me show you all of this live — about twenty minutes.

I'll build a project from nothing, push it to git, deploy dev and prod. Then *(beat)* I'll tamper with prod by hand — and we'll watch the check name the exact column, while GitHub shows nothing happened at all.

ORDER

- build → push → pull as an account clone → deploy dev, then prod from git → tamper prod → drift check → the un-reversible case.

16 Gate the data, not just the shape

BEYOND SCHEMA

SAY

One more thing, and it's the interesting part for us. This manages more than the **shape** of the tables.

The same declarative project can gate the **data** — quality checks declared right beside the schema. *(beat)* So it doesn't just prove the schema matches. It can prove the data passes.

IF ASKED

- We haven't used this in the POC — it's the highest-value next step. Validated it works in a quick spike (next slide is a real run).

17 Quality gates, declared beside the schema

BEYOND SCHEMA · EXAMPLES

SAY

Here's what that looks like. You attach a data metric function — no null IDs, unique IDs — with an **expectation**. Then one command, `snow dcm test`, runs them all.

This is a real run: one passed, one failed, with the exact value it saw. *(beat)* And it exits non-zero on failure, so it gates a pipeline. Bad data caught where it enters — not three dashboards later.

IF ASKED

- “Enterprise only?” — **No**. Verified working on Standard edition.
- Left side of an expectation is always the keyword `VALUE`. No subqueries, casts or arithmetic — a check, not a query.

SAY

Honestly: this is a proof of concept. One slice — eight tables, no data. Grants, tasks and streams we haven't exercised. But it proves the thing that matters.

(beat) The database can tell us, every morning, whether it still matches the repo — and name the column when it doesn't. We'd start with scheduled PLAN, which is read-only and safe. Deploy stays a human decision.

IF ASKED — NEXT STEPS

- Widen past capacities; rehearse the un-reversible recovery on a table with data; explore the data-quality expectations for real.

FACTS AT A GLANCE — QUICK REFERENCE

- **70** CREATE TABLE IF NOT EXISTS across **8** pipeline files — the problem
- **DCM GA:** 7 Aug 2026 (we built the POC during preview)
- **Edition:** works on **Standard** — schema and data-quality both verified
- **POC scope:** 8 tables, 53 columns, **no data**, personal account
- **Findings:** 13, four from deliberately breaking it
- **45 of 53** columns aren't last in their table → **ERROR** is the common case
- **Snowflake records DEPLOY, never PLAN;** native history 12 months, no **ACCOUNT_USAGE**
- **Nightly check:** 05:00 UTC, PLAN only, ~30 s, ran unattended two nights
- **Three verdicts:** CLEAN / DRIFT / ERROR
- **Cost:** warehouse compute only — a plan is ~3 s on an X-Small

QUESTIONS YOU'LL GET — AND YOUR ANSWERS

ADOPTION & COST

“What does it cost to run?” Warehouse compute only — a plan over 8 tables is about three seconds on an X-Small. No new licensing; it's a native GA feature. The drift log is trivial storage.

“How much effort to adopt for our real estate?” The DDL already exists — we capture it from the live database with `GET_DDL`, put it in `DEFINE` files, and version it. We diffed the capacities slice by hand and it matched exactly. The real work is process, not typing: make prod deploy from git, and schedule the nightly PLAN. Start PLAN-only — read-only, zero risk.

“Does this replace Matillion?” No. Matillion moves data; DCM manages the shape of the tables it lands in. DCM would own the DDL that today lives inside Matillion.

PRODUCTION & RISK

“Can we point this at production today?” The feature is GA and production-grade. Our POC is one slice with no data, so I'd go PLAN-first on prod — it's read-only, catches drift, and risks nothing — then bring DEPLOY under it once we've rehearsed the recovery paths. I would not schedule DEPLOY.

“DEPLOY drops columns — what about real data?” Exactly why DEPLOY is never automated — it's a human decision after reading a plan. Dropped-column data is recoverable via Time Travel within retention, but we treat a drop as deliberate and reviewed. Un-reversible cases (mid-list drop, narrowing a VARCHAR) are planned maintenance: unload, drop, redeploy, reload.

“What if two people deploy at once?” Honestly — untested. A genuine gap; I'd verify DCM's locking before relying on it in a team. Flagging it rather than guessing.

“How do we roll back?” Not a command — revert the commit and deploy again. Git is the rollback mechanism. Because prod is deployed from git, the previous good state is just an earlier commit.

“What if the nightly check silently stops?” That's the one we got wrong and fixed. A suspended task is silent exactly like a passing one, so the health view checks whether the task is actually *scheduled* — not just whether the log is recent — and says so loudly if it isn't.

ALTERNATIVES

“Why not Terraform?” Terraform suits account-level objects — warehouses, roles, integrations — and keeps an external state file you must manage and secure. DCM is native, stateless, and covers in-database objects. Common practice is both: Terraform for the account, DCM for the schema.

“Why not schemachange / migrations?” Those are imperative — you write the `ALTER` steps in order. DCM is declarative — you say what you want and it computes the diff. Migrations win for complex backfills; declarative wins for keeping shape in sync and detecting drift. You can use both.

“Why not just tell people not to edit prod by hand?” You can't enforce it, and emergencies happen — someone hotfixes prod at 2am. Discipline isn't a control. The point is to *detect* it when it happens so the repo can be corrected, not to trust it never does.

SECURITY & ACCESS

“Where does the GitHub token live?” In a Snowflake `SECRET` — encrypted, not readable back — referenced by the git integration. For production we'd rotate it and scope it to the specific repos.

“Who can deploy to prod?” Whoever holds the role that owns the project — deploy is role-gated like any privileged operation. And prod deploys come through git and review, so it's gated twice.

“Does DCM read our data?” No — it manages structure only, never rows. (Expectations run metric functions over data, but that's a separate, opt-in feature.)

SCOPE, MATURITY & OPS

“Is it GA / what edition?” GA since 7 Aug 2026; works on Standard. Tables, views, tasks, roles, warehouses, schemas are GA within DCM; streams and some policies are still preview.

“Does it scale to our whole estate?” Untested at our full size — the POC is 8 tables. A plan re-validates all definitions, and very large projects take longer. I'd widen incrementally and measure.

“Does it handle views, tasks, streams, grants?” They're supported object types — but we haven't exercised them in this POC. Tasks and grants are GA; streams are preview. Honest scope limit.

“How are we alerted, and how long is history kept?” Email via a notification integration, only when the verdict isn't clean (silent on clean, so nobody filters it). Our drift log is unlimited; Snowflake's native history caps at 12 months.

GLOSSARY — IN CASE A TERM COMES UP

- **DCM Projects** — Declarative Change Management: declare the desired schema, Snowflake reconciles to it.
- **DEFINE** — declares an object (vs `CREATE`, which instructs). The definition file is the source of truth.
- **PLAN** — a dry run; computes the changeset, changes nothing.
- **DEPLOY** — applies the changeset (`CREATE OR ALTER` underneath).
- **Drift** — the database differing from the declared state, from an out-of-band change.
- **GIT REPOSITORY** — an account-level clone of the repo inside Snowflake; updated by `FETCH`.

- **Workspace** — a per-user Snowsight editor connected to git; can't be read by automation.
- **env_suffix** — a templating value that appends `_DEV / _PROD` so one file builds both.
- **USING CONFIGURATION** — selects which config (dev/prod) a plan or deploy uses.
- **Expectation** — a data-quality check (metric function) attached to a table; run with `snow dcm test`.